

Project Documentation

AI WeatherWise: Intelligent Weather Forecasts and Insights

1. Introduction

- **Project Title:** AI WeatherWise – Deliver Intelligent Weather Forecasts and Insights
- **Platform:** Headless RESTful Backend API
- **Domain:** Environmental Data & Artificial Intelligence Services

2. Project Overview

- **Purpose:** AI WeatherWise is a RESTful backend service built to provide real-time weather metrics enriched with Google Gemini AI natural language processing. It solves the challenge of interpreting raw numeric weather data by delivering human-readable summaries and personalized activity and clothing recommendations.
- **Core Problem Addressed:**

Difficulty organizing and tracking multiple favorite locations across devices.

 - Lack of personalized recommendations for clothing and outdoor activities based on live weather conditions.
 - Cognitive overload from deciphering raw meteorological figures rather than actionable insights.
- **Key Features:**
 - Secure JWT authentication with bcrypt password hashing.
 - CRUD operations for managing favorite user locations.
 - Real-time weather data aggregation via OpenWeatherMap integration.
 - AI-powered summaries and custom advice via the Google Gemini SDK.
 - Built-in fallback mode to maintain API response availability during external service disruptions.
 - Centralized error handling and payload sanitization against NoSQL injection and XSS vectors.

3. Architecture & Technologies

- **Design Pattern:** Model-View-Controller (MVC)
 - **Model Layer:** Enforces data schemas, indexing, and validation via Mongoose ODM.
 - **Controller Layer:** Coordinates business workflows, API requests, weather fetching, and Gemini AI prompts.
 - **View (API Routing) Layer:** Headless routing endpoints mapping HTTP methods (**GET**, **POST**, **PUT**, **DELETE**) to controllers.
- **Runtime & Framework:** Node.js (v16+) with Express.js
- **Database:** MongoDB with Mongoose ODM
- **AI Engine:** Google Gemini SDK (@google/genai / @google/generative-ai)
- **Testing & Tools:** Postman, Thunder Client, Visual Studio Code

4. Setup Instructions

Prerequisites

- Node.js (v16 or higher)
- npm (v8 or higher)
- MongoDB instance (local or MongoDB Atlas)
- OpenWeatherMap API Key & Google Gemini API Key

Installation Steps

Bash

Clone the project repository

```
git clone <repository-url>
```

```
cd "AI Weatherwise"
```

Initialize and install dependencies

```
npm install express mongoose bcryptjs jsonwebtoken cors dotenv @google/genai
```

Install developer dependencies

```
npm install --save-dev nodemon
```

Environment Variables (.env)

```
PORT=5000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_jwt_secret_key
```

```
WEATHER_API_KEY=your_openweathermap_api_key
```

```
GEMINI_API_KEY=your_gemini_api_key
```

5. Folder Structure

AI-WeatherWise/

```
|— config/
|   |— db.js      # MongoDB connection logic
|— controllers/
|   |— authController.js # User registration and login
|   |— weatherController.js # Real-time metrics retrieval
|   |— aiController.js  # Gemini AI prompt orchestrations
|   |— locationController.js # Saved cities CRUD operations
|— middleware/
|   |— authMiddleware.js # JWT validation & role matrix shields
|   |— errorMiddleware.js # Centralized error handler & sanitization
|— models/
|   |— User.js      # User account schema
|   |— Location.js  # Favorite locations schema
|— routes/
|   |— authRoutes.js  # Authentication routes
|   |— weatherRoutes.js # Public & protected weather routes
|   |— aiRoutes.js    # Gemini recommendation routes
|   |— locationRoutes.js # City bookmark routes
```

```
|— .env          # Environment keys & configurations
|— .gitignore
|— index.js      # Server entry point & Express gateway
|— package.json
```

6. Database Schemas (Mongoose)

User Schema

```
const mongoose = require('mongoose');

const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, default: 'reader', enum: ['reader', 'admin'] },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('User', UserSchema);
```

Location Schema

```
const mongoose = require('mongoose');

const LocationSchema = new mongoose.Schema({
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  city: { type: String, required: true },
  country: { type: String, required: true },
  createdAt: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Location', LocationSchema);
```

7. Running the Application

```
# Start server in production mode
npm start
```

```
# Start server in development mode (with Nodemon)
npm run dev
```

Base URL: <http://localhost:5001>

Status Check: [GET http://localhost:5001/api/health](http://localhost:5001/api/health)

8. API Documentation

Method	Endpoint	Access	Description
POST	/api/auth/register	Public	Register a new user
POST	/api/auth/login	Public	Authenticate user and issue JWT
GET	/api/weather/current?city={city}	Public	Fetch real-time weather metrics
POST	/api/locations	Private (User)	Add a location to favorites
GET	/api/locations	Private (User)	Get all saved locations for user
DELETE	/api/locations/:id	Private (User)	Remove a saved location
POST	/api/ai/summary	Private (User)	Generate natural-language weather summary
POST	/api/ai/recommendations	Private (User)	Get Gemini-powered clothing/activity suggestions

9. User Roles & Flow

- **Admin:** Manages account health, inspects server activity, and audits database status.
- **Registered User:** Saves favorite cities, fetches personalized reports, and generates AI activity insights.
- **Public User:** Accesses basic, unauthenticated weather metrics.

End-to-End Workflow

1. User creates an account via `/api/auth/register` and logs in to receive a JWT.
2. User saves preferred travel destinations via `/api/locations`.
3. The platform retrieves current temperature, humidity, and precipitation data from OpenWeatherMap.
4. The system injects metrics into Google Gemini prompt templates to generate context-aware activity/clothing advice.
5. JSON response containing both structured weather telemetry and narrative AI advice is returned.

10. Testing & Validation

- **Tools Used:** Postman, Thunder Client
- **Test Cases:**
 - User authentication with valid/invalid credentials.
 - Route protection to confirm `401 Unauthorized` responses on missing JWT headers.
 - Resilient fallback verification when external API limits are exceeded or keys are absent.
 - Data schema validation ensuring duplicate emails and missing location parameters are rejected.

11. Known Issues

- High latency during external AI completions if the network encounters packet drops.
- OpenWeatherMap free-tier rate limits restrict rapid consecutive location queries.

12. Future Enhancements

- Push notifications and automated email alerts for severe weather warnings.
- Hourly micro-forecasts and radar map overlays via WebSocket connections.
- Automated travel packing lists generated based on 7-day weather predictions.
- Containerization using Docker and cloud deployment on AWS ECS or Google Cloud Run.